



FaultMorse: An automated controlled-channel attack via longest recurring sequence

Lifeng Hu^{a,b}, Fan Zhang^{a,b,c,d,e,*}, Ziyuan Liang^a, Ruyi Ding^f, Xingyu Cai^g, Zonghui Wang^a, Wenguang Jin^a

^a Zhejiang University, China

^b Key Laboratory of Blockchain and Cyberspace Governance of Zhejiang Province, China

^c ZJU-Hangzhou Global Scientific and Technological Innovation Center, China

^d Jiaxing Research Institute, Zhejiang University, China

^e Alibaba-Zhejiang University Joint Research Institute of Frontier Technologies, China

^f Northeastern University, United States

^g Baidu USA, United States

ARTICLE INFO

Article history:

Received 10 March 2022

Revised 28 October 2022

Accepted 1 November 2022

Available online 3 November 2022

Keywords:

Controlled-channel

Intel SGX

Operating system

Page fault

Automation

ABSTRACT

With the rise of the concept of Trusted Execution Environments (TEEs), such as Intel Software Guard Extensions (SGX), researchers are prompted to constantly verify its effectiveness. Controlled-channel attacks are proposed to construct side channels against the shielding systems by intentionally provoking page faults. So far, various powerful and noise-free controlled-channel attacks have been introduced. However, there are some challenges encountered in the actual practice of these attacks, e.g., extensive manual effort is always required to analyze the target binary and identify conditional control-flow patterns.

In this paper, we present *FaultMorse*, an automated controlled-channel attack. We adopt a global perspective to analyze the page fault sequence and find a specific recurring pattern that corresponds to some specific instructions in the program. Most of the secret bits can be automatically deduced by analyzing the locations of the recurring pattern in the page fault sequence. Compared to previous works, *FaultMorse* can reduce the complexity of analysis. We propose a method to control page fault counts to improve the attack performance. We implement our *FaultMorse* attack on a physical machine and evaluate its effectiveness, universality, and page-fault rate. The experimental results show that for some known vulnerable algorithms, *FaultMorse* can automatically deduce more than 99% of the secret bits.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

In the past years, the introduction of various security threats, such as malware, hardware Trojans (Xiao et al., 2016), kernel exploits, remote exploits, and even malicious cloud operators (Ion et al., 2011), poses a serious impact on the secure execution of applications that contain sensitive data and code. To defend against these threats, many hardware-based Trusted Execution Environments (TEEs) have been proposed to support *shielded execution*, such as the Trusted Platform Module (TPM) (Group, ARM's TrustZone (ARM, 2008), Samsung's KNOX (Samsung, 2013) and Intel's Trusted Execution Technology (TXT) (Futral and Greene, 2013), Software Guard Extensions (SGX) (Coporation, 2014). Inside these isolated environments, the strong security property makes the ex-

ecution of the sensitive program trustworthy, such that application code and data are protected against external entities.

The security of software applications heavily depends on the security of the underlying system software. With the rise of the concept of TEEs, the operating system (OS) is considered to be under the attacker's control and therefore untrusted. In traditional computing environments, when the OS is untrusted, the security of software applications is threatened. Comparatively, for TEEs such as Intel SGX, the root of trust is based only on the application code itself and the hardware implementation of the CPU. That is, even in a case where the underlying privileged software is untrusted, SGX can still ensure the confidentiality and integrity of an application. However, researchers have recently demonstrated *controlled-channel* attacks (Xu et al., 2015) against TEEs. The threat model assumes that the malicious OS kernel has near-total control over the platform. This control is used to construct powerful, noise-free side channels from enclave applications that rely on the kernel's

* Corresponding author.

E-mail address: fanzhang@zju.edu.cn (F. Zhang).

services. So far, various controlled-channel attacks (Gyselinck et al., 2018; Kim et al., 2019; Shinde et al., 2015; Van Bulck et al., 2018; 2017b; Wang et al., 2017) have been introduced to extract data inside enclaves by using page fault information. Despite the numerous defenses (Chen et al., 2018; 2017; Kim et al., 2019; Orenbach et al., 2020; Sasy et al., 2018; Shih et al., 2017; Strackx and Piessens, 2017) proposed by researchers, controlled-channel attacks have proven to be challenging to mitigate.

These controlled-channel attacks are challenging in practice and the adversary may encounter some difficulties - complexity, lack of information, and attack performance. Firstly, the residence of multiple functions on the same page makes the adversary require extra effort to identify which function is about to be accessed, which complicates the analysis. Secondly, the adversary does not always have knowledge of the target binary. In some scenarios such as Intel SGX Protected Code Loader (PCL) (Shacham), the enclave memory is protected, such that the adversary cannot tell virtual page numbers of those target functions in real-world attacks so it cannot be straightforwardly determined which memory pages to monitor. Lack of information about the locations of those target functions may raise difficulties and require additional manual analysis techniques. Thirdly, page-fault rate, one of the metrics to evaluate the attack performance, is also a challenge. That is, some academic defenses have been suggested to detect controlled-channel attacks, such as based on page-multiplexing compile-time transformations (Shinde et al., 2015) or run-time detection using Intel Transactional Synchronization Extensions (TSX) (Chen et al., 2017; Shih et al., 2017). In case of the latter, a large amount of occurrences of page faults may make the victim aware of the attack and adopt some mitigation techniques. However, due to excessive performance overheads or reliance on custom or repurposed hardware, these mitigations are purely academic research proposals and remain unused in practice. We argue that the page-fault rate can still be a valid scientific criterion to compare the performance of attacks and academic defenses.

In this paper, we present an automated controlled-channel attack, *FaultMorse*. We adopt a global perspective to observe the page fault sequence and find that there is a specific recurring pattern that corresponds to some specific instructions in the program. To identify and locate the recurring pattern in the page fault sequence, we implement a search algorithm based on the *suffix array* (Manber and Myers, 1990). We transform the secret extraction from the deduction on concrete page fault sequences to the deduction on the distance of adjacent iterative patterns, like in Morse code where different letters, numbers, and punctuation marks are deciphered by comparing different patterns. We only consider the page-granularity channel and find it sufficiently robust for certain attack scenarios. Instruction-granularity page fault information (Kim et al., 2019; Van Bulck et al., 2017b) (e.g., multiple repeated accesses to the same page) is out of the scope in our *FaultMorse* attack, which reduces the complexity of the analysis but also limits the classes of vulnerabilities that can be exploited. On the other hand, *FaultMorse* is designed to apply to multiple algorithms and implementations to show the benefit from automation. Note that *FaultMorse* targets a rather specific type of victim applications with loop iterations featuring conditional control flow. In addition, we propose control of the number of allocated memory pages for an enclave application to reduce the occurrences of page faults, such that the attack performance can be improved.

To distinguish from some early works (Kim et al., 2019; Shinde et al., 2015; Xu et al., 2015) that experimented on simulation platforms, we evaluate the effectiveness, universality, and page-fault rate of our *FaultMorse* attack on a physical machine. The results show that for some known vulnerable cryptographic algorithms, *FaultMorse* can automatically deduce more than 99% of the secret bits and has good applicability. Moreover, if the page-fault rate is

out of consideration, our *FaultMorse* attack requires only one execution trace to recover the secret completely. This applies more to the attack scenario where the adversary does not have permission to execute the victim program multiple times. In addition, we describe the method to control the occurrences of page faults while ensuring that it can succeed.

Contributions. The contributions of this paper can be summarized as follows.

- We propose *FaultMorse*, an automated controlled-channel attack to leak the runtime control flow from Intel SGX enclaves via automated page fault sequence extraction and analysis. We implement an attack program based on the *suffix array* to automatically search the recurring pattern. We release our experimental code of *FaultMorse* as open source.¹
- We propose a method to adjust the number of memory pages allocated to the enclave applications. It can limit the page-fault rate of the attack while ensuring the successful recovery of the secrets.
- We deploy our *FaultMorse* attack on a physical machine and evaluate its effectiveness with sufficient experiments. The results show that for some known to be vulnerable algorithms, *FaultMorse* can automatically deduce more than 99% of the secret bits. The comparative evaluation with previous works shows the good universality and the benefit from automation, as well as a lower page-fault rate.

Organization. The remainder of this paper is organized as follows. Section 2 provides the background on the SGX issues and controlled-channel attacks. Section 3 explains the threat model of our proposed attack. Section 4 describes the difficulties encountered in the existing work and states the core idea of our attack as well as the overview. Section 5 describes our proposed attack in more detail. Section 6 presents the implementation of our *FaultMorse* attack on the shielding system. Section 7 demonstrates the effects of our attack and shows the evaluation. Section 8 enumerates some related works. Section 9 discusses the strengths and weaknesses of our work and directions for future research. Finally, Section 10 concludes our work.

2. Background

In this section, we provide some necessary backgrounds on Intel SGX and page fault as well as some points about page access monitoring in SGX. Besides, we discuss previous research results on the controlled-channel attack.

2.1. EdDSA implementation

The Edwards-curve Digital Signature Algorithm (EdDSA) (Bernstein et al., 2012) is a high-security high-speed digital signature scheme using a variant of Schnorr signature based on a twisted Edwards curve. It is designed to work faster than existing digital signature schemes without sacrificing security. The security of elliptic curve public key cryptosystems is based on the computational uncontrollability of the elliptic curve discrete logarithm problem: given two points A and B on an elliptic curve, feasibly compute the scalar x , if it exists, such that $A = x \cdot B$.

EdDSA uses scalar point multiplication for public-key generation and the signing operation. Given a randomly generated large scalar value as the EdDSA secret k , the private key a is derived with a cryptographic hash function $a = \text{hash}(k)$, and the corresponding public key Q is calculated as $Q = s \cdot G$, with the base point G and part of the private key s . r is defined as the private session

¹ <https://github.com/Ezekiel-1998/FaultMorse>

key and (R, S) is defined as the signature of a message M , where $R = r \cdot G$ and $S = (r + a \cdot \text{hash}(R, Q, M))$. The long-term private key a can be directly recovered as $a = (S - r) / \text{hash}(R, Q, M)$ by an adversary who obtains the private session key r from side-channel observation during the process of signing (Bernstein et al., 2012; Yarom and Beniger, 2014).

Some vulnerabilities for EdDSA in the cryptographic library Libgcrypt v1.6.3 (Shinde et al., 2015), v1.7.5 (Van Bulck et al., 2017b) as well as v1.7.6 (Wang et al., 2017) have been reported. That is, although Libgcrypt provides protection against the side channel by marking the private key as “secure memory”, it indeed does not protect the session key r . Consequently, the non-hardened branch is always taken while the signature (R, S) is calculated, such that the session key may be disclosed.

2.2. SGX

Intel Software Guard Extensions (SGX) is a set of new processor architecture extensions to the x86 instruction set architecture. It is available on the majority of Intel processors and aims to protect the integrity and confidentiality of the code and data of sensitive applications in an untrusted execution environment. SGX helps the applications to be loaded and executed in a secure container called enclave, such that the code and data within an enclave are inaccessible to all external software, including the OS and hypervisor. In this case, sensitive applications can be effectively protected against various kinds of malicious entities outside the enclave, such as kernel-privileged malware.

Intel SGX reserves a range of continuous physical memory exclusively for enclaves called Enclave Page Cache (EPC), which is stored in Processor Reserved Memory (PRM). An enclave is usually initialized beforehand. The initial library or data is loaded into the enclave from an untrusted environment and a hash is generated after initialization. Then the user can verify that hash to check the integrity and correctness of the code in the enclave. In our scenario, some cryptographic libraries are placed into the enclave.

After initialization, even software with the highest privilege, such as OS and Virtual Machine Manager (VMM), cannot acquire any information inside the enclave. SGX relies on the Memory Encryption Engine (MEE) to decrypt enclave memory contents when entering the CPU package during the enclave execution and to encrypt them when the execution finishes. SGX provides a flexible programming model which allows an application to instantiate an enclave as part of its address space via the SGX instruction set. Consequently, no information about the address of memory content will be leaked. However, the untrusted OS can manage the page table responsible for address translation, which is vulnerable to attacks. The mapping of each virtual memory page to the physical memory, access permissions, cacheability, etc., can be controlled by the malicious OS.

2.3. Page fault and page access monitoring

Page fault is a type of exception raised by computer hardware when a running program accesses a memory page, which is not currently mapped by the Memory Management Unit (MMU) into the virtual address space of a process. Normally, the program used to handle this exception is a part of the OS kernel. If the OS determines that the access is valid, the page which is to be accessed requires a mapping to be added to the process page tables, and may additionally require the actual page contents to be loaded from a backing store such as a disk. And if the access is illegal, the OS usually terminates the related process.

Due to the fact that the page table and MMU are controlled by the OS, the OS can deliberately manipulate the permission of memory page access to generate page faults so that the faulting

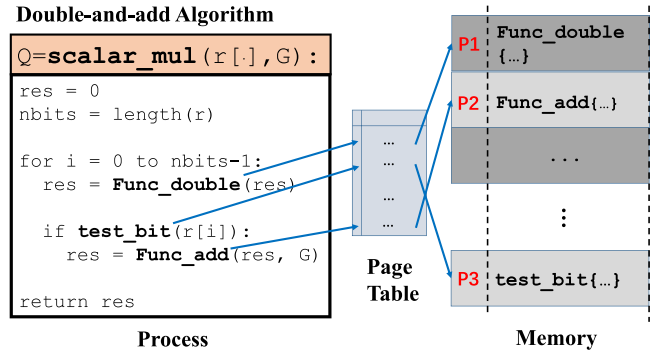


Fig. 1. Pigeonhole attack.

addresses can leak some sensitive information about applications inside the enclave. To prevent this attack, Intel SGX CPU only provides the faulting page number to the interrupt handler but conceals the exact memory address when such an exception occurs. However, work in Shinde et al. (2015) shows that the page fault channel is sufficiently strong to reveal sensitive data from unmodified versions of OpenSSL and Libgcrypt. These older versions of Libgcrypt attacked by prior works (Shinde et al., 2015; Van Bulck et al., 2017b; Wang et al., 2017) have now been patched in more recent versions of Libgcrypt (i.e., from Libgcrypt v1.7.7 on, using the patch contributed by Van Bulck et al., 2017b). However, to demonstrate the effectiveness of our attack, we use the same versions of Libgcrypt.

SGX prohibits the OS from obtaining fine-grained address information (e.g., instructions, cache lines) (Coporation, 2014), such that the OS cannot tell the exact address of the faulting instruction protected from external software. That is, the OS has difficulty identifying whether the page fault is triggered by accessing the monitored instruction, and thus may not configure its access permission at the right moment. As a result, page access monitoring in SGX has some difficulties in practice and it may require extensive manual effort to analyze in-depth.

2.4. Controlled-channel attack

The notion of controlled-channel attack is a term that Xu et al. (2015) coined in 2015. They first demonstrated a novel side-channel attack against shielding systems such as Overshadow, InkTag, and Haven. Since the OS manages the virtual-to-physical page translation tables for the sensitive application, it can observe all page faults and the faulting page numbers, which leak information. The untrusted OS uses its control over the platform to construct powerful side channels against the protected enclave that relies on services of the kernel. In the case of secret-dependent control flow or data accesses existing in the victim program, the malicious OS can observe the sequence of page faults which can be used to uniquely identify the specific memory access. Consequently, large amounts of sensitive information can be extracted from protected environments. However, this kind of attack relies on an exhaustive offline analysis of the target application binary to identify page fault sequences, which is impractical. Subsequent work by Shinde et al. (2015) which is called the Pigeonhole attack improved the previous work and demonstrated the power of the page fault channel. They extracted cryptographic key bits from unmodified versions of OpenSSL and Libgcrypt. An instance of the Pigeonhole attack is shown below.

The scalar point multiplication method is widely used in many asymmetric encryption algorithms, and the double-and-add algorithm (Knuth, 2014) is the most classical implementation of the scalar point multiplication method. Figure 1 shows the most com-

mon vulnerable part of this algorithm. The binary method computes Q , a point on the curve, with the scalar point multiplication $Q = \text{scalar_mul}(r[\cdot], G)$, where G is the base point. The scalar r is usually associated with the key and can be used to expose some sensitive information if leaked. When scanning $r[\cdot]$, a point doubling Func_double is always executed, but whether the point addition Func_add is executed or not depends on the function test_bit which tests the i th bit $r[i]$.

In the Pigeonhole attack, the adversarial OS intentionally allocates only a few memory pages to the victim process, thereby resulting in many page faults. The right side of Fig. 1 shows an illustration where three functions, the sub-routine for duplication Func_double , the sub-routine for addition Func_add and test_bit , are located in three different pages P_1, P_2, P_3 , respectively. If $r[i] = 0$, a sequence $[P_1 P_3]$ is observed. Otherwise, $[P_1 P_3 P_2]$ is observed, since the program calls an additional function Func_add located in page P_2 , and thus one more page fault P_2 appears at the end of the sequence. Therefore, each bit of $r[\cdot]$ can be extracted by the Pigeonhole attack using the page fault sequence.

3. Threat model

In this section, we explain the threat model of our proposed controlled-channel attack called the *FaultMorse* attack. Through this attack, the adversary is allowed to derive some sensitive information and data processed inside an enclave.

- The attack assumes the standard SGX threat model where the attacker has the authority to control the privileged system software, such as the OS. In this case, the malicious OS can manage (e.g., map and unmap) enclave memory pages but cannot see their contents. When a program in the enclave attempts to read or write a page which is not present in memory, the OS can receive as well as handle a page fault interruption.
- The controlled channel can contain only the page-granularity fault information while other side channels (e.g., cache, memory bus, instruction-granularity fault) are out of the scope for this paper. In our threat model, the information about the specific faulting address which requires more fine-grained attack techniques is unnecessary.
- The attack assumes only the knowledge of the algorithm for encryption or signature of the target application while the source code or binary is unnecessary.
- The attack targets a type of victim applications with specific characteristics whose loop iterations feature conditional control flow.

4. High-level view of the attack

In this section, we describe the challenges encountered in the actual practice of the existing work, such as the seminal work by Xu et al. (2015) and the subsequent work by Shinde et al. (2015). We state our core idea and give an overview of the *FaultMorse* attack which can overcome these difficulties.

4.1. Difficulties in practice

The success of the controlled-channel attack relies on the analysis of those collected page faults. In practice, when analyzing the actual page fault sequence, the adversary may encounter several categories of difficulties - complexity, lack of information, and page-fault rate.

Complexity. The first category is that the residence of multiple functions on the same page causes the complexity of the analysis. In computer architecture, a generic policy called *spatial locality* describes the default memory management policy that if the data

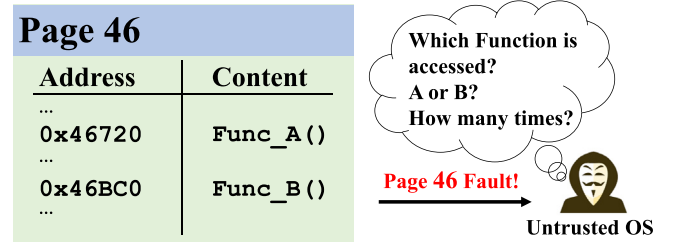


Fig. 2. High complexity of analysis.

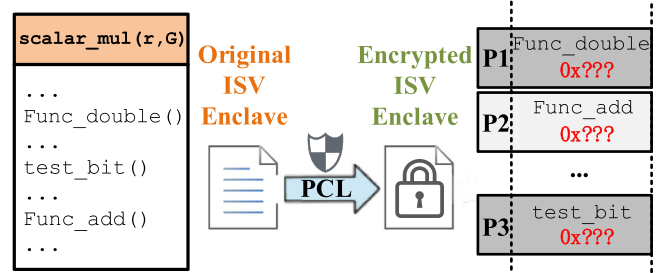


Fig. 3. Lack of knowledge of source code.

and code are continuous in the virtual space, they may co-locate on the same virtual page with a high probability. More precisely, they may be on the same physical page if the 4KB page boundary is not crossed. For instance, Fig. 2 shows that function Func_A shares the memory page 46 with another function called Func_B . When a page fault occurs on this memory page, it is hard for the malicious OS to identify which function is about to be accessed. In another case, when the functions Func_A and Func_B are invoked repeatedly and alternately, the OS can only intuitively observe one page fault on the first access to the memory page 46, and thus without extra effort, it can hardly tell how many times these functions have been called.

Some information can actually be exploited to analyze the page fault, such as the fact that the conditional control flow touches different code pages. That is, although the functions Func_A and Func_B are both located on page 46, they differ in the underlying functions and subsequent memory pages. Several prior works (Shinde et al., 2015; Wang et al., 2017) have deterministically exploited this property in the running example of libgcrypt to recognize the conditional control flow and recover the secret. In addition, more recent controlled-channel attacks (Kim et al., 2019; Moghimi et al., 2020; Van Bulck et al., 2017b) utilize the instruction counting techniques or finer-grained attack techniques to overcome the limitation. However, the existing controlled-channel attacks all require extensive manual effort to analyze the target binary and identify conditional control flow patterns to be exploited via a carefully crafted page-fault sequence.

Lack of information. The second category is that the attacker does not always have knowledge of the (compiled) source code of the target application. Intel SGX ensures the confidentiality and integrity of applications at run-time. However, it does not provide confidentiality of code offline as a binary file on a disk. To prevent the enclave from being reverse engineered, Intel SGX Protected Code Loader (PCL) (Shacham) is proposed to encrypt the enclave shared object (.so) at building time and decrypt it at enclave loading time. Under this threat model, even an adversarial OS does not deterministically learn the virtual page numbers of target function entries in real-world attacks. In a real scenario, the encryption library is loaded into an enclave beforehand, and the memory page information is hidden. Therefore, no extra information is leaked from exactly one page number or one pattern. As Fig. 3

shows, in the Pigeonhole attack scenario mentioned in the paper (Shinde et al., 2015), those functions `scalar_mul`, `Func_double`, `Func_add` and `test_bit` are the targets detected by the attacker to get the control flow of this program. However, the attacker has difficulty obtaining information masked by TEEs about the locations of those target functions (Coporation, 2014), which is the prerequisite for the attack. This raises difficulties and potentially requires additional manual analysis techniques. **Page-fault rate.** Another challenge is the page-fault rate of the attack. To perform a more effective attack, the malicious OS may allocate a low number of physical pages for the target enclave program to induce more page faults and get more side-channel traces. However, some academic mitigation solutions to the controlled-channel attack detect page faults and other synchronous exceptions as well as suppress those interrupts. For example, T-SGX (Shih et al., 2017) is proposed to monitor transaction aborts for signs of attacks and to terminate the program execution if the page fault incidence exceeds a set threshold. However, it relies on a commodity component of the Intel processor, called Transactional Synchronization Extensions (TSX), which is disabled on CPU generations from Skylake to Coffee Lake (Halfacree, 2021). Note that those mitigations remain purely academic research proposals that are not used in practice, due to excessive performance overheads or reliance on custom or repurposed hardware. It means that no satisfactory solution for production SGX enclaves exists to monitor the page-fault rate, but we argue that the page-fault rate could still become a valid scientific criterion to compare attacks and academic defenses. A high page-fault rate may increase the probability of victims perceiving the attack. For instance, the controlled-channel attack (Xu et al., 2015) requires millions of page faults and T-SGX sets the threshold value to 10 to defeat it. Thus, for attackers, consideration should be given to reducing the page-fault rate. However, reducing the incidence of page faults or interrupts while ensuring the effectiveness of the attack requires additional effort for analysis.

4.2. The core idea

To cope with the aforementioned insights that almost all existing controlled-channel attacks require considerable manual analysis, we propose a new method via automated page fault leakage extraction and analysis. Some previous work (Shinde et al., 2015; Xu et al., 2015) analyzed page faults with several specific target functions to get the control flow of the program. In contrast, we take a global perspective to analyze `for` loops in source code and observe that a fixed pattern is repeated multiple times in the page fault sequence. Instead of directly deriving a specific page fault sequence for a bit value of 1 or 0, the core idea of our trace analysis is that the special sequence pattern with a fixed number of recurrences is identified firstly. The secret extraction is transformed from the deduction on concrete page fault sequences to the deduction on the distance between adjacent iterative patterns.

Moreover, to improve the convenience and accuracy of our attack, a more effective approach is to seek the longest recurrent sequence, denoted as *LRS*, since its length is quite stable and has the least possibility of being affected by noise. Specifically, the *LRS* is the longest of all recurring subsequences of a long sequence, and its number of repetitions approximates the length of the secret. Just as different letters, numbers, and punctuation marks in Morse code are deciphered by comparing different patterns, we automatically identify each secret bit based on the distance between one *LRS* and the next, which leads to the feasibility, simplification, and enhancement of the attack in practice.

As Fig. 4 shows, the attacker monitors the execution of the double-and-add algorithm process mentioned in Section 2.4 and obtains a page fault sequence (in a real attack scenario, the sequence can be more complicated). By analyzing this trace, the *LRS*

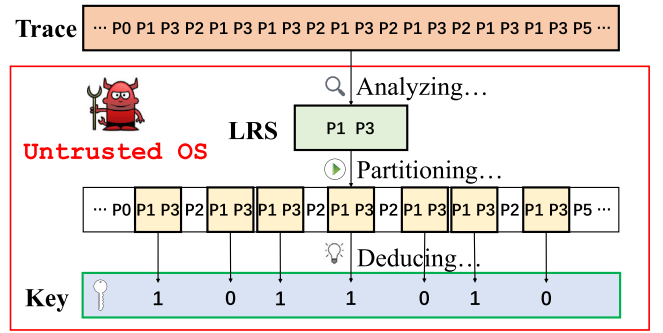


Fig. 4. Overview of the core idea.

is extracted, shown as *[P1 P3]*. The recurrent sequence indicates that each way of the branch calls the same functions which are located in these memory pages while the following sequence represents the difference in control flow between different ways of this branch. Then the *LRS* is used to partition the long trace. By comparing the difference in distance between two adjacent appearances of *LRS*, the attack process can automatically and deterministically deduce the bit values in the secret, while the information about the location of those vital functions in memory is unnecessary.

In terms of the page-fault rate of the attack, we find it unnecessary to induce as many page faults as possible to get more side-channel traces. We define the number of allocated physical pages for an enclave program as the size of the Pigeonhole set and denoted as λ . As λ increases, the number of page faults occurring over a period of time (the length of the page fault sequence) decreases. When λ increases to a certain extent, the number of page faults decreases a lot but is still sufficiently strong to extract the cryptographic secret bits. However, in this case, the attack succeeds at a much lower incidence of page faults.

4.3. Overview

As shown in Fig. 4, the proposed *FaultMorse* attack can be summarized as the following steps:

Step 1 Collecting page faults. The adversary $\mathcal{A}$ deliberately allocates limited memory pages to the victim process $\mathcal{V}$. Then $\mathcal{A}$ monitors the corresponding page fault from $\mathcal{V}$ and generates a sequence of multiple page faults, meanwhile reducing the interfered page faults possibly from other processes running in the same OS.

Step 2 Searching the *LRS*. The adversary $\mathcal{A}$ has to find and locate some specific patterns with a certain number of recurrences in the sequence. The technique of *suffix array* is used to search for the *LRS*.

Step 3 Inferring the scalar bits. With the identified *LRS*, the page fault trace is partitioned. The distance between two adjacent appearances of *LRS* can be used to infer the value (0 or 1) of most of the scalar bits in the secret or sensitive data.

Step 4 Cracking entire secret bits. Those small number of remaining secret bits are enumerated in an exhaustive searching manner. The correctness of the entire secret is verified by signing the message and comparing the signature, or encrypting the message and comparing the ciphertext.

5. The *FaultMorse* attack

In this section, we describe our proposed *FaultMorse* attack in more detail. Figure 5 shows the framework of the attack implementation. We choose the double-and-add algorithm (Knuth, 2014) mentioned in Section 2.4 as the object of analysis in this section.

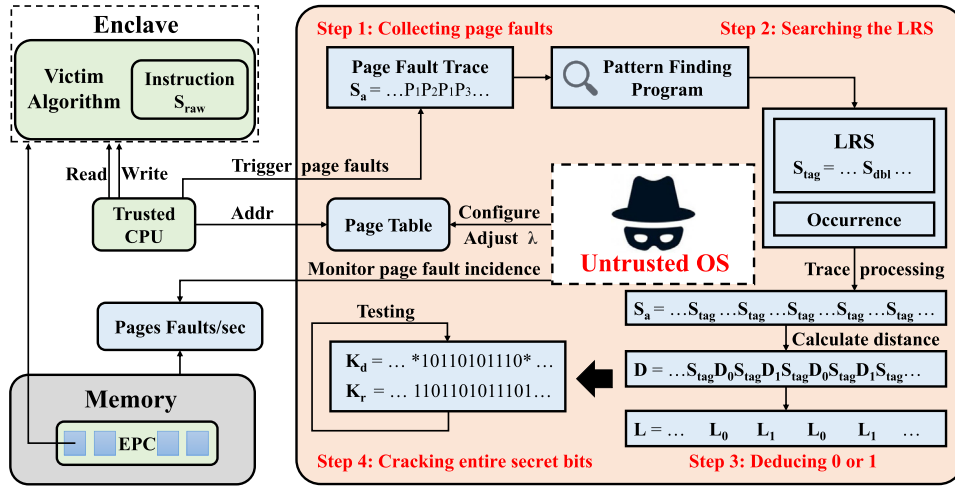


Fig. 5. Framework of the FaultMorse attack.

Our FaultMorse attack is applicable to a variety of algorithms, as long as the page access control flow is input-dependent. That is, the control to either P_1 or P_2 is dependent on the secret bit, just like the `if` branch of the function `scalar_mul` shown in Fig. 1.

5.1. Collecting page faults

As the adversarial operating system, $\mathcal{A}$ first sets the size of the pre-allocated memory (in pages) for the victim enclave $\mathcal{V}$. In the simplest case, λ is set as 1, which is also discussed in Wang et al. (2017). The binary code of the target program is loaded as instructions, denoted as S_{raw} , into the single page. If two consecutive instructions are located on two different pages, a page fault is raised and triggers an interrupt, then the adversarial OS $\mathcal{A}$ is informed. Suppose instruction I_1 and I_2 are located in page P_1 and P_2 respectively. After P_1 is loaded into the memory page allocated for $\mathcal{V}$, I_1 is executed. Then the OS fetches I_2 and finds it on another page from P_1 . A page fault is raised and sent to $\mathcal{A}$, allowing P_1 to be swapped out and P_2 to be swapped in. This raised page fault is tagged with the page number of P_2 . In this way, a page fault trace, S_a , is generated, where $S_a = P_1, P_2, \dots$.

In the case mentioned above, we set λ as 1 which means the malicious OS allocates only one memory page for the target enclave program. This makes the attack easily observable because whenever the functions called by two adjacent instructions are not on the same memory page, a page fault is generated to inform the OS and interrupt the program process. As Wang et al. (2017) shows, they performed the page fault attack against Libgcrypt (v1.7.6) to recover the EdDSA session keys where they controlled the size of accessible memory as one page. Consequently, they found that for secret bit 1 and 0, there were 89 and 48 subsequent page visits respectively, and in total around 71,000 page faults were triggered. The generation of a large number of page faults may lead victims to perceive that they are under attack if they take some steps to detect malicious spy processes. To reduce the page-fault rate of the attack, we use the memory counter (Pages Faults/sec) to monitor the page fault counts and adjust the size of pre-allocated memory based on it.

When λ is set greater than 1, S_a is determined by the operating system's page replacement policy. Typically, for out-of-date system instructions, the computer uses three kinds of algorithms to evict it from memory, FIFO (First In First Out), LRU (Least Recently Used), and LFU (Least Frequently Used). All of these three algorithms are used in modern computer systems. In this paper, we design the pre-allocated memory using LRU as its basic algorithm for mem-

ory scheduling, since it is almost the most widely used in practice. If a memory page has been accessed recently, the possibility of being accessed again may be high in the future. The idea of LRU is to evict the least recently requested memory pages to minimize the occurrence of page faults. Consequently, if we consider the pre-allocated memory for the victim enclave $\mathcal{V}$ as a queue, the length of the queue is equal to λ . Whenever a page is accessed, its page number is moved to the head of the queue, and all other page numbers in the queue are shifted back one bit. When the queue is full, the page at the end of the queue is discarded. The adversarial OS collects the page faults when the memory page to be accessed is not recorded in the queue. But if we choose other basic algorithms for memory scheduling, such as FIFO, we may obtain a different page fault trace. When the queue is full, the memory page to be evicted is the first accessed one instead of the least recently requested one. Thus, more page faults may be raised and the collected page fault sequence is also different, which means the scheduling algorithm is relevant for our attack.

5.2. Searching the LRS

As mentioned in Section 4.1, those existing controlled-channel attacks require extensive manual effort or carefully crafted page-fault sequences to deduce the bits of $r[\cdot]$. However, the promise of FaultMorse is to prevent that via automated page fault sequence analysis. The rolled-loop structure of the calculation process of the double-and-add algorithm could be used for analyzing the session key differently.

Note that the function `Func_double` is always executed, regardless of the specific value of $r[i]$, as shown in Fig. 1. The specific page fault sequence for `Func_double`, denoted as S_{dbl} , is complicated to deduce, since `Func_double` may also reside in the page of `scalar_mul` and `Func_add`. However, when the program calls the function `Func_double` in each loop, the specific page fault sequence S_{dbl} should be identical, and the number of occurrences of S_{dbl} , denoted as N_{dbl} , should be ideally approximate to the length of the scalar, i.e., $N_{dbl} = \text{length}(r)$. Consequently, it is reasonable to assume that there exists a particular recurring pattern, denoted as S_{tag} , which has an approximate number of occurrences of $\text{length}(r)$. It can also be assumed that S_{dbl} is a substring of S_{tag} with the same number of occurrences.

To identify S_{tag} as well as count and locate those occurrences, a search algorithm based on suffix array is adopted. Suffix array (Manber and Myers, 1993) is an array obtained by sorting all suffixes of a given string. This incredibly powerful data structure is

often used for string searches, and it has some advantages over *suffix tree*, such as improved space requirements. We implement a program using *suffix array* that takes a long sequence as input for analysis and outputs the longest recurring substring in this sequence. In this program, for a given page fault sequence *Str* of length *n*, we convert the hexadecimal string to a decimal representation and define a string array *Suffix[n]* to record all the suffixes of *Str*. To reduce the time complexity, we use the DC3 algorithm (Kärkkäinen et al., 2006) to sort all the suffixes in *Suffix[n]* according to their initial letters, and put the indices of the starting positions of these sorted suffixes into the suffix array *Sa[n]* in order. That is, *Sa[i]* denotes the index of the starting position of the suffix which is ranked *i* among all suffixes in *Suffix[n]*. Based on *Sa[n]*, we can quickly build the rank array *Rank[n]*, which records the ranks of all suffixes. That is, *Rank[i]* denotes the rank of *Suffix[i]* among all suffixes. Then we build the height array *Height[n]*, which records the lengths of the longest common prefix of two suffixes adjacent to the rank. That is, *Height[i]* denotes the length of the longest common prefix between *Suffix[Sa[i]]* and *Suffix[Sa[i - 1]]*. Obviously, the maximum value in *Height[n]* records the length of the longest recurring substring in the long sequence. And we eventually search for positions of the longest recurring pattern in the long sequence according to the index of $\text{MAX}(\text{height}[n])$. In addition, we need to add some restrictions in the search process to avoid some cases. For example, in Fig. 4, the longest subsequence that repeats at least twice may be simply found, i.e., $[P1\ P3\ P2]$, which is not expected. Thus we set a higher priority search requirement, i.e., the searched longest recurring pattern needs to be repeated approximately $\text{length}(r)$ times.

We input the page fault trace found in the previous step into this program and get the *LRS* efficiently, although the trace is always very long. According to the inference above, this *LRS* contains S_{dbl} and represents that the algorithm program calls the function *Func_double*. At this point, we can distinguish the corresponding position of each loop in the sequence.

5.3. Deducing 0 or 1

There exist two categories of patterns in the *for* loop - the scalar bit is 0 or 1, denoted as L_0 and L_1 , respectively. To distinguish L_0 and L_1 , we have to measure the length of the page fault sequence of every loop, i.e., we should calculate the distance between two adjacent S_{tags} found in previous steps, denoted as D . Since an additional function *Func_add* is executed when the scalar bit is 1, a longer distance can be found between a S_{tag} of L_1 and the next S_{tag} , denoted as D_1 . And when the scalar bit is 0, a shorter distance can be found between a S_{tag} of L_0 and the next S_{tag} , denoted as D_0 . Based on this, for each secret bit, we can deterministically deduce whether it is 1 or 0 by comparing D . In this way, we can get $\text{length}(r) - 1$ bits of the key at most, denoted as K_d , except the last bit, since there is no more target pattern at the end of the sequence to calculate its distance. That is, theoretically, we can deduce exactly 511 bits of a hidden 512-bit secret.

5.4. Cracking entire secret bits

For the remaining unknown secret bits Bit_m , we infer K_r as the full key based on K_d with methods of exhaustion. To determine whether our attack is successful, we use K_r to encrypt the plaintext. When the ciphertext matches the result encrypted in the enclave system, our attack is considered successful.

In the real attack scenario, we commonly find two missing bits, with one at the beginning and the other at the end of the K_d . For the former, the program usually requires some initialization operations when processing the first bit of the secret so that the page fault sequence of the first loop may be irregular compared

Table 1
Experiment setup.

Module	Configuration
Processor	Intel(R) Core(TM) i7-6770HQ CPU, 2.60 GHz
Memory	8 GB, 2133 MHz
Disk	240 GB, LITEON T10 240 SSD
Operating System	Ubuntu 16.04 server 64 bits

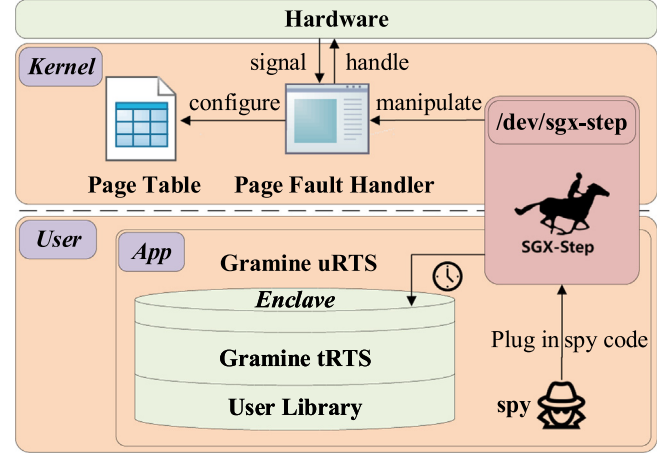


Fig. 6. Gramine attack framework interaction.

to S_{tag} . For the latter, the last bit of the secret can never be inferred as 1 or 0, since there is no more S_{tag} that can be used to calculate the distance between two loops. Ideally, we reduce the key searching space to a point where only two missing bits remain, i.e., $\text{length}(\text{Bit}_m) = 2$. Consequently, we set a threshold τ as $\text{length}(r) - 3$ to determine whether the attack is successful.

6. Implementation

In this section, we present the experimental setup of our FaultMorse attack implementation on the shielding system as well as the target algorithm in this scenario.

We run the experiment on an Intel NUC6i7KYK. Table 1 lists the experiment setup.

Framework. Figure 6 shows the framework of our attack implementation. Similar to previous controlled channel attacks (Shinde et al., 2015; Van Bulck et al., 2017b; Wang et al., 2017; Xu et al., 2015), our attack targets unmodified legacy applications running inside the isolated environment where the *shielded execution* protects the application binary inside the enclave from the untrusted host OS. As with the previous work Van Bulck et al. (2017b), we implemented our FaultMorse attack on the open-source Graphene-SGX library OS (Tsai et al., 2017). Graphene-SGX (which has now been renamed to Gramine Tsai et al., 2022) is a lightweight library OS which repackages conventional OS kernel services into a user-mode application library. Gramine is designed to run a single application with minimal host requirements and it can run applications in an isolated environment with benefits comparable to running a complete OS in a virtual machine - including guest customization, ease of porting to different OSes, and process migration. We implement our attack as an extension of Gramine's untrusted runtime, without changing the trusted in-enclave components. It can be used as a reusable framework to enable eavesdropping on different application binaries. **Collecting page faults.** As shown in Fig. 6, to collect the page fault information, we use SGX-Step (Van Bulck et al., 2017a) as a framework to construct the attack. SGX-Step consists of a Linux kernel driver and user space libraries that allow attackers

to configure untrusted page table entries and interrupt x86 APIC timers directly from the user space environment. Our spy code can run in user space, utilizing SGX-Step's functionality to edit PTEs and register the page fault handler. Before starting the experiment, we load the application binaries and their support files such as the software library into the SGX enclaves. In addition, we use SGX-Step to construct and maintain an array to record the virtual page numbers of those memory pages which are placed in the allocated memory. Only memory pages recorded in the array are marked as executable in the page table, while other memory pages whose page numbers are evicted from the array are marked as non-executable. When the victim program inside the enclave starts to execute, some memory pages are swapped out (they are marked as non-executable and page faults will occur on the next access to these pages) and others are swapped in (a signal of page fault is reported and triggers an interrupt, then the spy process configures the page table to mark these pages as executable). To minimize noise, we pin the victim process to a specific CPU core and use a signal handler within the victim process, such that the false alarms from other processes are ignored. Once a page fault is raised, the spy process receives the signal and executes the `Page_fault_handler()` to record the page fault and configure the page table. In this way, the untrusted OS can automatically and accurately collect the page fault information and perform the FaultMorse attack to recover the key or encrypted message.

Targets of attack. To illustrate the universality of our FaultMorse attack mentioned in Section 5, we choose some real-world applications as the target algorithms under attack in the evaluation. More specifically, we perform the FaultMorse attack to reproduce some previously published controlled-channel attacks (Shinde et al., 2015; Van Bulck et al., 2017b; Wang et al., 2017) on a general purpose cryptographic library Libgcrypt, showing that our attack can automatically extract the same information. As in the previous work, we choose Libgcrypt EdDSA as the main algorithm to evaluate the effectiveness of FaultMorse, and the results of some other algorithms for the attack are also documented in Section 7.

7. Evaluation

In this section, we demonstrate the effectiveness of our proposed FaultMorse attack and evaluate the attack framework by automatically extracting sensitive information from protected SGX enclaves. We design the experiment to extract private EdDSA session keys from unmodified binaries of the widely used cryptographic library Libgcrypt mentioned above. In addition, we attack against some other algorithms in Libgcrypt as well, to evaluate the applicability to multiple real-world applications. Finally, we demonstrate the method to adjust the page-fault rate and evaluate the performance of FaultMorse.

7.1. Effectiveness

We conduct sufficient experiments with different to-be-signed messages M and EdDSA secrets k to illustrate the effectiveness of our FaultMorse attack. In this phase, the value of λ is fixed. In detail, we set λ to 8 and randomly generate a large number of M and k . The processes of deriving the long-term private key $a = \text{hash}(k)$ and calculating the corresponding public key $Q = a \cdot G$ are invisible to the spy. The signature process inside the enclave generates the private session key r by hashing a together with M , and it signs one message with one session key at a time. The signature (R, S) which is calculated as the tuple $(R = r \cdot G \text{ and } S = (r + a \cdot \text{hash}(R, Q, M)))$ is explicit to the spy. The OS acts as an adversary to execute the FaultMorse attack outside the enclave. If the attacker discloses the session key r from the controlled-channel

Table 2

Summary of the effectiveness of FaultMorse attack with changing to-be-signed messages and EdDSA secrets.

Variable: Message			
Pattern length	Occurrence	Count	Average $D_0 \setminus D_1$
66, 68, 71, 72	510	4	140 \ 332
≤ 70	511	127	139 \ 328
71	511	187	146 \ 336
72	511	215	143 \ 331
73	511	207	145 \ 333
74	511	154	142 \ 335
≥ 75	511	106	149 \ 342
Variable: Secret			
Pattern length	Occurrence	Count	Average $D_0 \setminus D_1$
70, 73	510	2	138 \ 331
≤ 70	511	125	137 \ 329
71	511	157	143 \ 327
72	511	191	142 \ 335
73	511	205	138 \ 330
74	511	183	140 \ 332
≥ 75	511	137	144 \ 338

observation during the process of signing and recovers the long-term private key a with the known (R, S) , Q , and M , we consider the attack successful.

Figure 7 presents the detailed result of an attack. In this attack, we restrict the number of memory pages to get the page fault sequence and input it into the program which can search for the *LRS* of this sequence. As the figure shows, we get the *LRS* [38, 17B, 17C, 179, ...] as well as its positions in the sequence. For instance, an *LRS* appears at the 438th number of this page fault sequence, which indicates that an iteration of a `for` loop is executed here. Besides, we learn the length of this pattern is 72 and the number of occurrences is 511, which indicates that except for the last bit, one more missing bit needs to be deduced with paying extra effort. We calculate distances between one *LRS* and the next, and obtain two kinds of results, i.e., 143 and 331, which represent an iteration of a `for` loop where the scalar bits are 0 and 1 respectively. Consequently, as inferred from the above, we can deterministically deduce the value of the secret, i.e., [010110010...].

Table 2 lists the results of the FaultMorse attack against Libgcrypt v1.6.3 EdDSA. For the two scenarios of changing messages and secrets, we perform the attack 1000 times to give a statistical evaluation. Table 2 enumerates all possible *LRS* lengths and counts their appearances in the 1000 experiments, as well as calculates the corresponding average values of D_0 and D_1 . In the case of the λ being fixed to 8, we find that the pattern length, as well as D_0 and D_1 , are almost not changed. The fourth column indicates that in the case of the corresponding pattern length, the calculated average values of D_0 and D_1 can be distinguished. The extremely obvious difference between D_0 and D_1 makes it easy to deduce each bit of the secret value. In addition, we obtain the result that the number of occurrences of *LRS* is almost always 511, which indicates that the Bit_m is 2. As explained in Section 5, there are almost always two missing bits, with one at the beginning and the other at the end of K_d , thus such an attack is considered successful, depending on the value of τ we set. In the remaining 6 of these 2000 experiments, we find that the last *LRS* also has a probability of not being found, and the number of occurrences of *LRS* is 510. Based on the experimental results, the success rate is 99.7% and we can deterministically illustrate the effectiveness of our FaultMorse attack.

7.2. Universality

To evaluate the applicability on multiple algorithms and implementations, FaultMorse reproduces some of the real-world at-

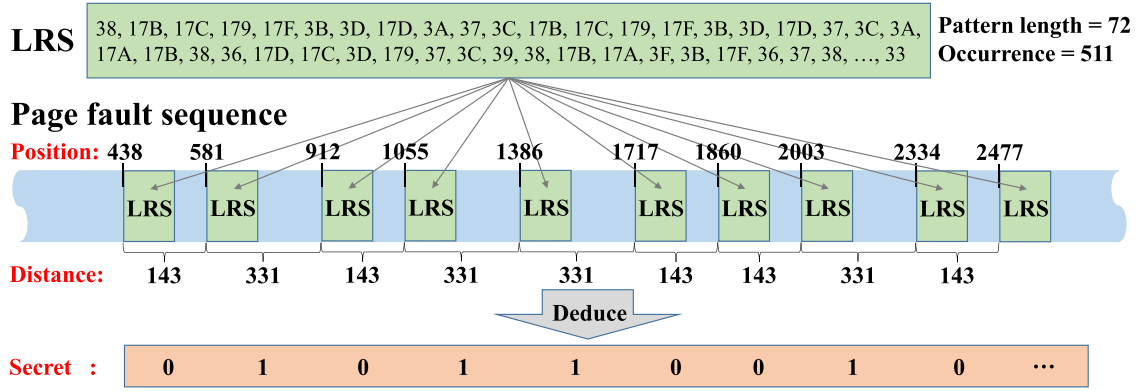


Fig. 7. An instance of the FaultMorse attack experiment.

Table 3

Summary of the applicability of FaultMorse attack on multiple algorithms of different Libcrypt versions.

Libcrypt	Algorithm	Vulnerable Function	Input Bits	Deduced Bits	%	Search Time (s)
v1.6.3	EdDSA	_gcry_mpi_ec_mul_point	512	510	99.61	198
	RSA	_gcry_mpi_powm	2048	2046	99.90	513
	ElGamal	_gcry_mpi_powm	400	398	99.50	121
	DSA	_gcry_mpi_powm	256	254	99.22	65
v1.7.5	EdDSA	_gcry_mpi_ec_mul_point	512	510	99.61	210
v1.7.6	EdDSA	_gcry_mpi_ec_mul_point	512	511	99.80	264

tacks from prior work and shows the benefits from automation. We perform FaultMorse attack against the unmodified binaries for Libcrypt v1.6.3 (Shinde et al., 2015), v1.7.5 (Van Bulck et al., 2017b) and v1.7.6 (Wang et al., 2017). Note that these are older versions of Libcrypt that have been attacked by these cited works, while the patch contributed by Van Bulck et al. (2017b) has now been applied to more recent versions (starting with Libcrypt v1.7.7). Table 3 summarizes the results of our study. Note that for the vulnerable function `_gcry_mpi_powm`, we add compilation option `-DUSE_ALGORITHM_SIMPLE_EXPONENTIATION` to use the old implementation. The results show that for those algorithms with proven vulnerabilities in libcrypt, FaultMorse can automatically deduce more than 99% of the secret bits. In most cases, except for the first and last bits, the rest of the secret bits can be recovered correctly. The rightmost column of Table 3 indicates the time required to search for LRS automatically, which is within the acceptable range.

7.3. Page-fault rate

To evaluate the impact of adjusting λ on the page-fault rate of the attack, we control the to-be-signed message and EdDSA secret to remain unchanged and increase λ step by step. In this subsection, we also attack against Libcrypt v1.6.3 EdDSA. As shown in the left diagram in Fig. 8, with the growth of λ , the length of the page fault sequence decreases rapidly, which indicates that the information leakage drops rapidly. This helps to reduce the page-fault rate and enhance the performance of the attack, thus reducing the probability of the attack being detected to some extent.

Impacts of λ . We study the influence on the effectiveness of FaultMorse when we only change λ . There are two kinds of potential impacts. Firstly, the increase of λ leads to the reduction of page fault counts, and at a certain point, we cannot find a suitable pattern for the attack. For instance, when we set λ to 32, the length of the page fault sequence is reduced to around 50, and we can hardly get valid information. Secondly, the increase of λ reduces the discrimination between D_0 and D_1 , which creates more difficulties for our deduction process. As shown in the middle diagram in Fig. 8, the values of D_0 and D_1 as well as their difference

Table 4

Summary of the evaluation of page fault counts, the length of LRS, D_0 , D_1 dropping with increasing λ .

λ	LRS	D_0	D_1	Page fault counts	Success
2	352	468	995	375122	✓
3	292	396	840	316928	✓
8	72	143	331	121836	✓
26	12	36	37	18748	✓
27	10	27	28	14169	✓
28	8	21	21	10760	✗
29	7	16	16	8213	✗
32	0	0	0	48	✗

keep decreasing with λ increasing. Hence, we confirm that λ can affect the discriminability of the attack, but there is still a clear boundary between D_0 and D_1 , such that we can distinguish them and deduce the secret.

Table 4 evaluates how page fault counts as well as the length of LRS, D_0 and D_1 drop as λ increases. To make the results more explicit, we only record the page fault counts during the execution of the vulnerable function `_gcry_mpi_ec_mul_point` instead of the complete EdDSA signature. These results make sense because as λ increases, instructions can be fetched to more pre-allocated physical memory pages, which obviously reduces the incidence of page faults. These numbers show that when we set λ less than 28, the length of D_0 and D_1 can always be distinguished, which indicates the success of FaultMorse. But when λ is set greater than 28, no pattern seems to be available, since D_0 and D_1 tend to be identical. In this case, we cannot distinguish whether each secret bit is 1 or 0, so the FaultMorse fails. And when λ is set greater than 32, we can not even search for LRS. This is because when Libcrypt EdDSA executes the for loop of the function `_gcry_mpi_ec_mul_point`, all associated instructions are located in these 32 pre-allocated physical memory pages, such that no pages are swapped in and out, i.e., no page fault is raised. Note that those 48 page faults are raised

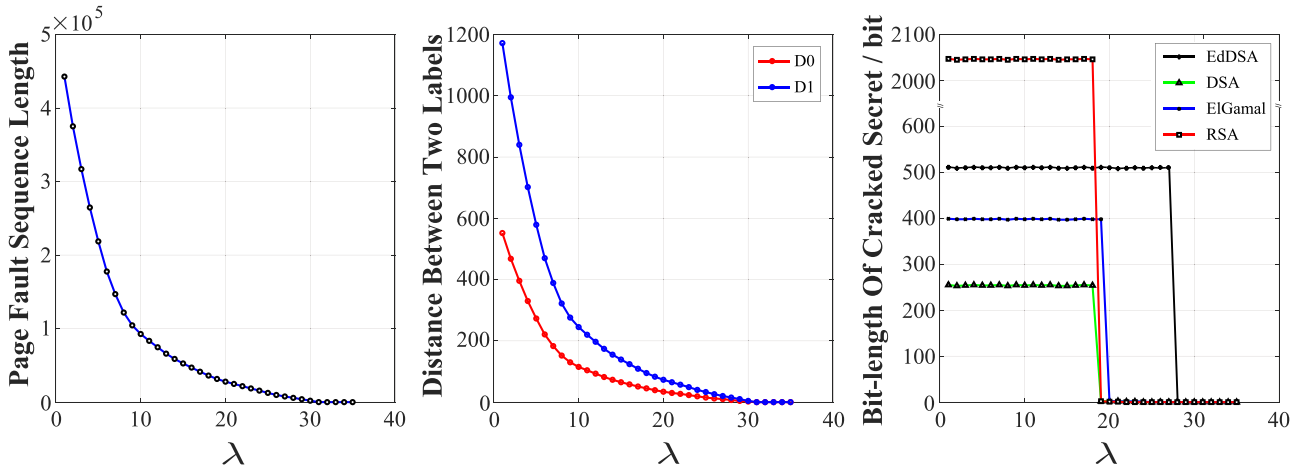


Fig. 8. Variation of page fault sequence length, distance between two labels and bit number of cracked secret.

before and after the execution of the `for` loop, independent of the `for` loop process.

Comparative Evaluation. In the best case, *FaultMorse* sets λ to 27 and triggers 14,169 page faults to recover the full secret, while a normal execution of EdDSA signature without attack only incurs about 1,500 Asynchronous Enclave Exits (AEXs) (Wang et al., 2017). Furthermore, Table 4 shows that only 28 pages would need to be allocated to prevent leakage. And during the execution of the `for` loop of `_gcry_mpi_ec_mul_point`, it can even prevent any swapping from occurring when 32 pages are available. However, even on SGXv1 machines with 128MB EPC, there would be roughly 32,000 available 4KB pages, which is much more than the memory pages needed for defense. Thus we acknowledge that such an amount of page faults in the condensed time and space are likely to be detected by any hypothetical advanced page-fault rate monitoring defense systems. However, by comparison, most previous works for real-world Libgcrypt attacks report a larger number of interrupts. Van Bulck et al. (2017b) reports that they attack EdDSA and recover the full 512-bit session key by monitoring the Access and Dirty bits. Their A/D attack on Libgcrypt v1.6.3 interrupts the victim enclave about 60,000 times and they count about 40,000 inter-processor interrupts (IPIs) for the attack on Libgcrypt v1.7.5. In addition, their cache-based attack on Libgcrypt interrupts about 130,000 times for a single start-to-end run. Wang et al. (2017) reports that they perform the page fault attack on Libgcrypt 1.7.5 and trigger around 71,000 page faults to correctly recover all the session secret bits. Furthermore, in total around 33,000 interrupts are triggered by B-SPM attacks and around 1,300 interrupts are triggered by T-SPM attacks. However, these two kinds of attacks require more side channels, such as the trigger page number and the leakage of time, which are unnecessary and out of the scope for our attack. As previously analyzed, provoking fewer page faults or interrupts could become a valid goal to enhance the performance of the attack. Based on the comparative evaluation, *FaultMorse* has a lower page-fault rate and better performance.

Properly Controlling λ . In general, we confirm that to-be-signed messages and EdDSA secrets have a slight influence while λ is the key factor to the success of *FaultMorse*. Properly controlling λ allows the attacker to recover the secret value accurately and also ensures the page-fault rate of the attack, since increasing λ can reduce page fault counts. As mentioned above, there are two kinds of potential impacts on the *FaultMorse* attack when we only adjust λ . Both of these two impacts can disable our *FaultMorse* attack. The right diagram in Fig. 8 shows the number of bits of K_d cracked by our *FaultMorse* attack for each value of λ . For Libgcrypt EdDSA, we find that the result always keeps smooth and stays close to 512, but when λ is greater than 27, its value decreases rapidly

and eventually converges to 0. It is because we cannot distinguish between D_0 and D_1 to deduce the secret bits, and there is no pattern repeated for around 512 times can be found. It is crucial for the *FaultMorse* attack to choose the value of λ rationally.

The threshold value of λ for determining whether the *FaultMorse* attack succeed is not always equal to 27. As shown on the right diagram in Fig. 8, we also evaluate the impact of adjusting λ on multiple algorithms and implementations mentioned in Table 3, and find they have almost the same downward trend and each of them has a specific sharp decline. It indicates that each of these algorithms has its own specific threshold value, and *FaultMorse* fails when the set value of λ is greater than the threshold value. The threshold value depends on the algorithmic program under attack. During the execution of the `for` loop, when the difference in the number of instructions between L_0 and L_1 grows, the difference in the number of memory pages accessed between L_0 and L_1 also grows. At the same time, the difference in page fault counts between D_0 and D_1 also increases, such that the value of λ can be scaled up while D_0 and D_1 can still be distinguished, i.e., the attack still succeeds but has a lower page-fault rate. Therefore, before *FaultMorse* is performed, the attacker first needs to generate some random secrets and messages as input for signing to measure the threshold of the algorithm. Then the value of λ is controlled to approximate the threshold value, improving the performance of the controlled channel by minimizing page fault counts while the attack succeeds in recovering the secrets.

If the adversary has permission to execute the victim program only once, λ should be set as 1 to induce as many page faults as possible. This ensures the effectiveness of the attack, while the page-fault rate is out of consideration.

Potential Improvements. Since the source code or binary of the target algorithm is not available for offline analysis in our threat model, the configuration process would need to be executed online by interacting with the victim enclave and cause a large amount of page faults, especially when starting with low values of λ . To address this challenge, we consider offline attacks on multiple algorithms in multiple cryptographic libraries and complete the configuration process in advance to construct a database on λ , such that the online analysis process may be simplified. This serves as a potential improvement and requires long-term maintenance of the database.

8. Related work

In this section, we discuss some important studies which are related to the *FaultMorse* attack.

Controlled-channel attack. Xu et al. (2015) first proposed the concept of the controlled-channel attack, which was a typical instance of the memory mapping manipulation attack. Their attack leveraged the intentional page faults due to memory access by the application. Attackers could get the information about secret-dependent data access and control flow in the target applications running on top of the SGX-based Haven Baumann et al. (2015) architecture. They implemented the attack on Haven and InkTag, and revealed complete text documents and outlines of JPEG images inside the enclave to demonstrate its power. However, their attack required exhaustive offline analysis of the target application binaries to identify page fault sequences, and then used this information to extract sensitive information without noise during the victim program's runtime. These requirements are difficult to reach in actual attack situations.

Pigeonhole attack. Pigeonhole attack is the subsequent work of the controlled-channel attack by Shinde et al. (2015), which demonstrates that the page fault side-channel is sufficiently strong to reveal sensitive information inside the protected enclaves. They implemented the attack to extract bits of encryption keys from commodity implementations of cryptographic routines in OpenSSL and Libgcrypt, such as AES, EdDSA, DSA, RSA, and CAST5. The principle and an instance of pigeonhole attack have already been mentioned in Section 2. Moreover, we mention the challenges encountered in the actual practice in Section 4.

Page table-based attack. To solve the challenge that multiple instructions may reside in the same memory page, Van Bulck et al. (2017b) proposed a technique which can infer the page access information of enclaves even without causing page faults. Compared to the page fault channel, their attack explicitly monitored the page table entries (PTE) in real-time to establish whether they have been accessed or not. To overcome the challenge that the subsequent accesses to the same page cannot be observed by the spy thread, they chose to observe the PTE status such as the Access and Dirty bits. Once the data and code inside the enclaves are accessed, page read/write attempts can be inferred by monitoring the changes in the A/D of the PTE. Attackers use cache attack techniques and Inter-Processor Interrupt to instantly identify the repeated memory accesses. Compared to the previous attacks, it requires cache-level techniques and side-channel information.

Version IDentification attack. Kim et al. (2019) extended the discussion of the page table-based attack (Van Bulck et al., 2017b). They demonstrated that the information (e.g., the code algorithm, SDK library version, configuration) inside the SGX enclaves is no longer hidden securely, since the page granularity of fault monitoring is broken. Based on the development that each instruction inside the same page can raise distinguishable faults, the attacker can obtain more detailed fault information. They proposed the Version IDentification (VID) attack against SGX and conducted experiments to evaluate its effectiveness. First, they extract page access pattern information of various algorithms of libraries, and set up a database with these universally known control-flow patterns. When an algorithm from the publicly known libraries inside the SGX enclave is executed, the attacker can deduce and identify the algorithm by extracting the fine-grained page access pattern and finding the matching pattern in the database. However, the VID attack is based on the assumption that the attacker can count the number of executed instructions inside the same page which stems from the technique by Van Bulck et al. (2017b). Besides, the accuracy of the VID attack is not guaranteed, which is a limitation.

9. Discussion

In this section, we discuss some drawbacks of our proposed attack and directions for future research.

9.1. Current attack

Limitation. For some algorithm implementations designed with defensive measures, the FaultMorse attack may fail. Since FaultMorse exploits secret-dependent control flow, it can be prevented, similar to other side-channel attacks, by implementing proper constant-time code, as is widely considered best-practice for cryptographic algorithms. For instance, the w-ary non-adjacent form (wNAF) method is used to improve the computational efficiency of the scalar multiplication method, where the points $\{1, 3, 5, \dots, 2^{w-1} - 1\}P$ and their negatives are pre-computed. Since these points are put into the same array, they have a high probability of being stored on the same memory page. When a fault occurs on this memory page, it is difficult for the attacker to determine which point is used, so it can only be confirmed that this window is not empty, but cannot deduce its exact value. When attacking the sliding window technique for exponentiation mentioned in Shinde et al. (2015), FaultMorse also only partially recovers the secret, i.e., the leakage depends on the input.

Impact from hardware/compiler. For the technology of SGX2.0 which breaks the memory restriction and provides larger EPC memory space, our FaultMorse attack is still effective. Because the system framework does not change where the malicious OS still owns near-total control over the platform. It can rewrite the page table to construct the controlled channel to obtain sensitive information. In addition, when compiling cryptographic libraries with different kinds and versions of compilers (GCC, LLVM, ICC), the allocation of instructions on physical memory varies, which may affect the efficiency of FaultMorse.

9.2. Future work

Subsequent development. As mentioned in Section 5, FaultMorse is applicable to multiple algorithms and implementations, as long as their page access control-flow is input-dependent. It indicates that FaultMorse has good universality which has been evaluated in Section 7. Based on this advantage, FaultMorse might be developed as a detection tool for some real-world cryptographic libraries or user-implemented algorithms. FaultMorse can at least initially detect whether the program control-flow is vulnerable to information leakage about the memory page access, such that it can inform users to patch the vulnerability as soon as possible. This will be our main research direction in the future.

Optimization. In terms of the page-fault rate of FaultMorse, we evaluate the impact of adjusting λ and compare our results with some previous works. Although Section 7 shows that FaultMorse requires fewer page faults or interrupts compared to some prior attacks, it may be still easily detectable. We believe FaultMorse has still room for improvement in page-fault rate. When λ is set to 28, D_0 and D_1 being equal causes us to not be able to deduce each secret bit. But at this point, they are not equal to zero, which indicates that some memory pages that are not helpful for the attack occupy these pre-allocated memories. FaultMorse may be able to trigger fewer page faults if it ignores these memory pages. In this regard, our future research work is to design a memory page monitoring policy that automatically filters some memory pages when collecting page fault information.

Applicability. As discussed above, it is not tricky to fix the vulnerabilities in these cryptographic algorithms. And these patches have been available in their new versions. However, we hypothesize that our FaultMorse attack also has potentials to be applied to some general-purpose enclave applications. We choose one of the widely used applications, FreeType (fre), which was studied in the original controlled-channel attack paper (Xu et al., 2015), as the object of our discussion on the applicability of the FaultMorse attack. The work in Xu et al. (2015) reports that in

this user-level font library, different glyphs have different hints and commands, such that control flows for rendering them depend on the character. When FreeType executes commands and processes hit to draw a glyph onto a bitmap, the attacker can exploit these control flows to infer the rendered text. In addition, the work in Xu et al. (2015) shows that two sets of page-fault sequences are identified for inferring the start and end of the render function `TT_Load_Glyph`. Once a character is rendered with its glyph, the function `TT_Load_Glyph` is invoked and these two sets of page-fault sequences can be observed. So they could be exactly the *LRS* mentioned in our attack. That is, if we perform our *FaultMorse* attack when users render text onto bitmaps, we can obtain a long page fault sequence, and the found *LRS* of this sequence is probably one of the identified patterns. In such a way, the number of repetitions of *LRS* is roughly equal to the number of function invocations, i.e., the number of characters. The work in Xu et al. (2015) also shows that the number of page faults over an invocation of `TT_Load_Glyph` uniquely identifies the character being rendered. Thus *FaultMorse* could also identify the character by calculating and comparing the distance between two adjacent *LRS*s. Performing *FaultMorse* attack against FreeType and some other enclave applications is also planned to be explored in our future work.

10. Conclusion

In this paper, we discuss the challenges encountered in the actual practice of recently introduced memory mapping manipulation attacks. Existing controlled-channel attacks all require extensive manual effort to analyze the target binary and identify conditional control-flow patterns via a carefully crafted page fault sequence. To overcome the challenges - complexity of the analysis and lack of information, we propose to adopt a global perspective to automatically observe and analyze the page fault sequence. After obtaining the *LRS*, we can deduce most of the secret bits by analyzing the distance between two adjacent labels. On the other hand, we propose a method to reduce page fault counts, such that the attack has better performance and can still succeed. We evaluate the effectiveness, universality, and page-fault rate of *FaultMorse* on a physical machine, and demonstrate that our attack applies to multiple algorithms and can automatically deduce more than 99% of the secret bits.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Lifeng Hu: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing. **Fan Zhang:** Writing – review & editing, Supervision, Project administration. **Ziyuan Liang:** Data curation, Writing – original draft, Visualization. **Ruyi Ding:** Conceptualization. **Xingyu Cai:** Supervision. **Zonghui Wang:** Supervision. **Wenguang Jin:** Supervision.

Data Availability

Data will be made available on request.

Acknowledgement

This work was supported in part by National Key R&D Program of China (2020AAA0107700), by National Natural Science Founda-

tion of China (62227805, 62072398), by Alibaba-Zhejiang University Joint Institute of Frontier Technologies, by Zhejiang Key R&D Plan (2021C01116), by Leading Innovative and Entrepreneur Team Introduction Program of Zhejiang (2018R01005), and by Research Institute of Cyberspace Governance in Zhejiang University, by National Key Laboratory of Science and Technology on Information System Security, by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Key Laboratory of Cyberspace Situation Awareness of Henan Province.

References

- ARM, 2008. Building a secure system using trustzone technology.
- Baumann, A., Peinado, M., Hunt, G., 2015. Shielding applications from an untrusted cloud with haven. *ACM Trans. Comput. Syst. (TOCS)* 33 (3), 1–26.
- Bernstein, D.J., Duif, N., Lange, T., Schwabe, P., Yang, B.-Y., 2012. High-speed high-security signatures. *J. Cryptogr. Eng.* 2 (2), 77–89.
- Chen, G., Wang, W., Chen, T., Chen, S., Zhang, Y., Wang, X., Lai, T.-H., Lin, D., 2018. Racing in hyperspace: closing hyper-threading side channels on SGX with contrived data races. In: 2018 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 178–194.
- Chen, S., Zhang, X., Reiter, M.K., Zhang, Y., 2017. Detecting privileged side-channel attacks in shielded execution with Déjà Vu. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, pp. 7–18.
- Coporation, I., 2014. Intel software guard extensions programming reference. https://01.org/sites/default/files/documentation/intel_sgx_sdk_developer_reference_for_linux_os.pdf.pdf.
- Freetye. <http://www.freetype.org/>.
- Futral, W., Greene, J., 2013. Intel Trusted Execution Technology for Server Platforms: A Guide to More Secure Datacenters. Springer Nature.
- Group, T. C. Trusted platform module (TPM) summary. <https://trustedcomputinggroup.org/resource/trusted-platform-module-tpm-summary/>.
- Gyselink, J., Van Bulck, J., Piessens, F., Strackx, R., 2018. Off-limits: abusing legacy x86 memory segmentation to spy on enclaved execution. In: International Symposium on Engineering Secure Software and Systems. Springer, pp. 44–60.
- Halfacree, G., 2021. Intel sticks another nail in the coffin of TSX with feature-disabling microcode update. https://www.theregister.com/2021/06/29/intel_tsx_disabled/.
- Ion, I., Sachdeva, N., Kumaraguru, P., Čapkun, S., 2011. Home is safer than the cloud! privacy concerns for consumer cloud storage. In: Proceedings of the Seventh Symposium on Usable Privacy and Security, pp. 1–20.
- Kärkkäinen, J., Sanders, P., Burkhardt, S., 2006. Linear work suffix array construction. *J. ACM (JACM)* 53 (6), 918–936.
- Kim, D., Jang, D., Park, M., Jeong, Y., Kim, J., Choi, S., Kang, B.B., 2019. SGX-LEGO: fine-grained SGX controlled-channel attack and its countermeasure. *Comput. Secur.* 82, 118–139.
- Knuth, D.E., 2014. Art of Computer Programming, Volume 2: Seminumerical Algorithms. Addison-Wesley Professional.
- Manber, U., Myers, G., 1990. Suffix arrays: a new method for on-line string searches. In: Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms. Society for Industrial and Applied Mathematics, USA, pp. 319–327.
- Manber, U., Myers, G., 1993. Suffix arrays: a new method for on-line string searches. *SIAM J. Comput.* 22 (5), 935–948.
- Moghim, D., Van Bulck, J., Heninger, N., Piessens, F., Sunar, B., 2020. Copycat: controlled instruction-level attacks on enclaves. In: 29th {USENIX} Security Symposium ({USENIX} Security 20), pp. 469–486.
- Orenbach, M., Baumann, A., Silberstein, M., 2020. Autarky: closing controlled channels with self-paging enclaves. In: Proceedings of the Fifteenth European Conference on Computer Systems, pp. 1–16.
- Samsung, 2013. White paper: an overview of samsung Knox.
- Sasy, S., Gorbunov, S., Fletcher, C.W., 2018. ZeroTrace: oblivious memory primitives from intel SGX. NDSS.
- Shacham, L., Intel(r) software guard extensions (SGX) protected code loader (PCL) for linux® OS. <https://github.com/intel/linux-sgx-pcl>.
- Shih, M.-W., Lee, S., Kim, T., Peinado, M., 2017. T-SGX: eradicating controlled-channel attacks against enclave programs. NDSS.
- Shinde, S., Chua, Z. L., Narayanan, V., Saxena, P., 2015. Preventing your faults from telling your secrets: defenses against pigeonhole attacks. *arXiv preprint arXiv:1506.04832*.
- Strackx, R., Piessens, F., 2017. The heisenberg defense: proactively defending SGX enclaves against page-table-based side-channel attacks. *arXiv preprint arXiv:1712.08519*.
- Tsai, C.-C., Porter, D. E., Vij, M., 2022. Gramine library os with intel sgx support. <https://github.com/gramineproject/gramine>.
- Tsai, C.-C., Porter, D.E., Vij, M., 2017. Graphene-SGX: A practical library {OS} for unmodified applications on {SGX}. In: 2017 {USENIX} Annual Technical Conference ({USENIX}{ATC} 17), pp. 645–658.
- Van Bulck, J., Piessens, F., Strackx, R., 2017. SGX-Step: a practical attack framework for precise enclave execution control. In: Proceedings of the 2nd Workshop on System Software for Trusted Execution. Association for Computing Machinery, New York, NY, USA doi:10.1145/3152701.3152706.
- Van Bulck, J., Piessens, F., Strackx, R., 2018. Nemesis: studying microarchitectural timing leaks in rudimentary cpu interrupt logic. In: Proceedings of the

- 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 178–195.
- Van Bulck, J., Weichbrodt, N., Kapitza, R., Piessens, F., Strackx, R., 2017. Telling your secrets without page faults: Stealthy page table-based attacks on enclave execution. In: 26th {USENIX} Security Symposium ({USENIX} Security 17), pp. 1041–1056.
- Wang, W., Chen, G., Pan, X., Zhang, Y., Wang, X., Bindschaedler, V., Tang, H., Gunter, C.A., 2017. Leaky cauldron on the dark land: understanding memory side-channel hazards in SGX. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 2421–2434.
- Xiao, K., Forte, D., Jin, Y., Karri, R., Bhunia, S., Tehranipoor, M., 2016. Hardware trojans: lessons learned after one decade of research. *ACM Trans. Des. Autom. Electron. Syst. (TODAES)* 22 (1), 1–23.
- Xu, Y., Cui, W., Peinado, M., 2015. Controlled-channel attacks: deterministic side channels for untrusted operating systems. In: 2015 IEEE Symposium on Security and Privacy. IEEE, pp. 640–656.
- Yarom, Y., Benger, N., 2014. Recovering OpenSSL ECDSA nonces using the FLUSH+reload cache side-channel attack. *IACR Cryptol. ePrint Arch.* 2014, 140.



Lifeng Hu received his BEng on information engineering from Zhejiang University, China, in 2020. He is currently pursuing the master's degree in the College of Information Science and Electronic Engineering, Zhejiang University, China. His research interests include system security and side-channel attack.



Fan Zhang received his PhD degree from the Department of Computer Science and Engineering, University of Connecticut, USA. He is currently a full professor in School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, and affiliated with Alibaba-Zhejiang University Joint Research Institute of Frontier Technologies. His research interests include hardware security, system security, cryptography, and computer architecture.



Ziyuan Liang (Graduate Student Member, IEEE) is currently pursuing the PhD degree in information science and electronic engineering with the School of Cyberspace Research, Zhejiang University, Hangzhou, China. His research interests include secure computing, data security, secure hardware, and side-channel attacks.



Ruyi Ding is a second-year PhD candidate at Northeastern University, MA, US. His research interests include hardware security, side-channel attack, and deep learning security. This work was done while Ruyi Ding was in Zhejiang University under the supervision of Prof. Fan Zhang.



Dr. Xingyu Cai is now a research scientist in Baidu Research USA. He received his PhD from University of Connecticut in Computer Science and Engineering in 2020. Before that, he obtained his Master and Bachelor degrees from Xi'an Jiaotong University, China. He has a broad research interests on applied data mining and machine learning algorithms. In particular, he works on topics related to natural language and speech processing, time series and sequential data analysis. His work has been published in top peer-reviewed conferences such as NeurIPS, ICLR, etc, as well as high impact journals.



Zonghui Wang is a senior engineer in the college of Computer Science and Engineering at Zhejiang University in Hangzhou, China. He received his PhD in the college of Computer Science and Engineering at Zhejiang University in 2007. His current research interests focus on computer architecture, system security, embedded system and cloud computing.



Wenguang Jin received his PhD in the college of Biomedical Engineering and Instrument Science at Zhejiang University in 2003. He is currently an associate professor in the college of Information Science and Electronic Engineering at Zhejiang University in Hangzhou, China. His research interests include wireless wearable human-computer interaction technology and devices, wireless communication and embedded systems.